

repo + file	role	workload	thread_model	primitives	tradeoff_notes
alacrity/alacrity:alacrity/src/logging.rs	Logging system	I/O-bound	Shared-state	Mutex, Arc<AtomicBool>	Formatting happens before locks so only actual writes are serialised, reducing lock hold time while still protecting shared I/O state
alacrity/alacrity:alacrity/src/renderer/mod.rs	Rendering	Coordination-heavy	Initialisation / singleton	AtomicBool, OnceLock<T>	Shared mutable state is minimised through one-time initialisation and immutable shared caches, avoiding runtime locking costs
alacrity/alacrity:alacrity_terminal/src/event_loop.rs	I/O coordination layer	Mixed	Resource arbitration model	Arc<FairMutex<T>>, Arc<T>	Fair locking prevents renderer starvation during heavy PTY throughput, but introduces additional synchronisation and coordination overhead
alacrity/alacrity:alacrity_terminal/src/synchronisation.rs	Synchronisation	Coordination-heavy	Resource arbitration model	parking_lot::Mutex<T>	Provides strong fairness guarantees and starvation prevention, but every lock acquisition requires layered coordination through an additional mutex
alacrity/alacrity:alacrity_terminal/src/os_integration.rs	OS integration bridge	Coordination-heavy	Actor / message-passing model	Arc<T>, Arc<Mutex<Option<T>>>	Avoids blocking inside WinAPI callbacks through atomics
dani-garcia/vaultwarden:src/api/icons.rs	Caching layer	I/O-bound	Async executor-driven coordination	AtomicPtr<T>	while limiting mutex use to mutable registration state
dani-garcia/vaultwarden:src/config.rs	Shared configuration	Coordination-heavy	Shared-state	AtomicBool, RwLock<T>	Read-heavy configuration access is fast and scalable, but replacing whole snapshots makes updates coarse-grained
dani-garcia/vaultwarden:src/db/mod.rs	Database	Coordination-heavy	Resource arbitration	Arc<T>, OwnedSemaphorePermit, Arc<Mutex<Option<T>>>	Blocking Diesel work is isolated from the async runtime, but semaphore and mutex coordination can create contention
dani-garcia/vaultwarden:src/http_client.rs	coordination	heavy	model	OnceLock<DbConnType>	under heavy DB load
dani-garcia/vaultwarden:src/main.rs	Resource coordination layer	I/O-bound	Initialisation / singleton	LazyLock, Arc, Mutex	Shared clients and resolvers reduce expensive reconstruction overhead, but the regex cache introduces a small global contention point
helix-editor/helix:helix-dap/src/transport.rs	Runtime coordination layer	Coordination-heavy	Async executor-driven	Arc	Arc-based ownership sharing avoids excessive locking, but coordination responsibilities are distributed across runtime tasks and framework boundaries
helix-editor/helix:helix-event/src/redraw.rs	Request coordination	Coordination-heavy	Actor / message-passing model	Arc, tokio::sync::Mutex	Mutex protects correlation state safely across async tasks but the shared pending-request table can become a contention hotspot under heavy debugger traffic
helix-editor/helix:helix-event/src/registry.rs	Rendering	Coordination-heavy	Shared-state	parking_lot::RwLock	Guarantees redraw correctness and synchronisation ordering but rendering can stall while long-running async readers hold the lock
event/src/registry.rs	coordination layer	heavy	concurrency model	RuntimeLocal<RwLock<Registry>>	Central registry simplifies extensibility and dispatch coordination but shared handler maps require locking and manual type-erasure safety management
		heavy	passing model	, RwLock	

helix-editor/helix:helix-event/src/runtime.rs	Runtime coordination layer	Coordination- heavy	Shared-state concurrency model	parking_lot::RwLock	Runtime isolation prevents cross-runtime interference but lazy initialisation paths require additional lock coordination and indirection
helix-editor/helix:helix-term/src/application.rs	Execution loop	Coordination- heavy	Async executor-driven coordination	Arc, ArcSwap	Avoids extensive shared-memory locking through async coordination and snapshot sharing, but correctness depends on event ordering and message flow
helix-editor/helix:helix-view/src/document.rs	State synchronisation layer	Mixed	Snapshot / copy-on-write model	ArcSwap, Arc, parking_lot::Mutex, Weak	Snapshot-based sharing gives scalable concurrent reads and async consistency but incremental remapping and transactional coordination logic become highly complex
rustdesk/rustdesk:src/client.rs	Session coordination	Coordination- heavy	Shared-state concurrency model	Arc, RwLock, Mutex	Centralised shared session state simplifies coordination across networking, IPC, and UI subsystems, but correctness depends on disciplined lock usage
rustdesk/rustdesk:src/client/io_loop.rs	I/O coordination layer	Mixed	Producer-consumer / pipeline coordination model	Arc, RwLock, Atomic	Separates ownership transfer from shared runtime telemetry, improving streaming throughput, but coordination between async tasks and worker threads is difficult to reason about
rustdesk/rustdesk:src/rendezvous_mediator.rs	Connection	Coordination- heavy	Shared-state concurrency model	AtomicBool, Mutex, RwLock, Arc	Shared coordination state enables robust NAT traversal and failover correctness, but distributed lifecycle management depends heavily on correct atomic ordering and locking discipline
rustdesk/rustdesk:src/server.rs	Runtime coordination layer	Coordination- heavy	Actor / message-passing model	Arc, RwLock, Mutex, AtomicBool, Weak	Shared runtime registry centralises service lifecycle management, but global shared state increases coordination complexity across services and worker threads
topjohnwu/Magisk:native/src/core/daemon.rs	Runtime coordination layer	Coordination- heavy	Shared-state concurrency model	OnceLock, Mutex, AtomicBool, AtomicArc	Daemon acts as central authority with shared mutable runtime, separation between immutable config and synchronised mutable subsystems, state that is performance sensitive uses atomics instead of locks, module metadata uses write-once/read-many to avoid synchronisation after boot
topjohnwu/Magisk:native/src/core/logging.rs	Logging system	I/O-bound	Producer-consumer / pipeline coordination model	Mutex, AtomicI32, Arc	High-performance logging design: synchronisation protects handle ownership, not IO itself, it uses Arc to avoid serialised writes, uses POSIX PIPE_BUF guarantees instead of userspace locking, separation between daemon logging path and zygisk path
topjohnwu/Magisk:native/src/core/su/daemon.rs	Caching layer	Coordination- heavy	Shared-state concurrency model	AtomicArc, Mutex, Arc	Fast path: lock-free cache reuse, slow path: rebuild policy from DB -> atomically publish shared object. Locking around mutable authorisation state -> small contention. Arc allows request handlers to retain policy snapshots.

topjohnwu/Magisk:src/core/thread	Resource coordination	Coordination-	Worker-thread pool	Mutex, Condvar, LazyLock	Only one shared task slot exists, so task submission is serialised, synchronisation separates task execution from lock ownership, condition variables coordinate state transitions
.rs	layer	heavy	execution model		
	State synchronisation	I/O-bound	Producer-consumer / pipeline	arc, OnceLock, AtomicUsize	Debouncing and batching reduce duplicate sync work, but suppression counters and async ordering make behaviour harder to reason about
farion1231/cc-switch:src-tauri/src/comman	layer		coordination model		
	Runtime coordination	I/O-bound	Shared-state	arc,rwlock	Centralised shared services simplify cross-component access, but global state increases coupling and requires careful async lock management
farion1231/cc-switch:src-tauri/src/lib.rs	layer		concurrency model		
	I/O coordination layer	Coordination-	Shared-state	arc,rwlock, AtomicU32	Atomics reduce contention for counters, while coordinated state transitions still require async locking and careful visibility guarantees
farion1231/cc-switch:src-tauri/src/proxy/ci		heavy	concurrency model		
	Request coordination	Mixed	Async executor-driven coordination	arc,mutex, AtomicBool	Streaming responses remain responsive because persistence is detached into async tasks, but lifecycle coordination across streams and logging is complex
farion1231/cc-switch:src-tauri/src/proxy/re	layer				
	Resource coordination	I/O-bound	Shared-state	arc,rwlock	Shared long-lived routing and failover state improves consistency across requests, but increases coupling between subsystems
farion1231/cc-switch:src-tauri/src/proxy/se	layer		concurrency model		
	Event dispatch system	Coordination-	Resource arbitration	tokio::sync::Mutex, Arc	Single async mutex guarantees consistency for sync operations, but serialises uploads/downloads and may delay concurrent requests
farion1231/cc-switch:src-tauri/src/services,		heavy	model		
	Rendering	Coordination-	Shared-state	Rc, RwSignal	Reactive snapshot updates minimise recomputation overhead for UI rendering but the design is restricted to single-threaded UI execution
lapce/lapce:lapce-app/src/editor_tab.rs	coordination layer	heavy	concurrency model		
	State synchronisation	I/O-bound	Producer-consumer / pipeline	RwSignal, Arc, AtomicU64	Asynchronous plugin coordination reduces blocking during installs and updates but maintaining consistent UI state requires careful lifecycle and ordering management
lapce/lapce:lapce-app/src/plugin.rs	layer		coordination model		
	Event dispatch system	Coordination-	Shared-state	Arc, Mutex, AtomicU64	Pipeline-style batching and event routing improve responsiveness but coordinating many independent event producers increases orchestration complexity
lapce/lapce:lapce-proxy/src/dispatch.rs		heavy	concurrency model		
	Connection	I/O-bound	Actor / message-	Arc, Mutex	Channel-driven coordination minimises shared mutable state but correctness depends strongly on RPC ordering and synchronisation between streams
lapce/lapce:lapce-proxy/src/plugin/lsp.rs	coordination layer		passing model		
	Request coordination	Coordination-	Actor / message-	Arc, Mutex, AtomicU64	Mailbox-style coordination keeps most request flow asynchronous but the shared pending-request map can still become a central synchronisation bottleneck
lapce/lapce:lapce-rpc/src/core.rs	layer	heavy	passing model		
	Resource coordination	I/O-bound	Resource arbitration	Arc,RwLock	Shared pooled HTTP clients reduce TCP/TLS setup overhead, but the global RwLock introduces coordination overhead during initialisation and reset paths
nushell/nushell:crates/nu-command/src/nt	layer		model		

nushell/nushell:crates/nu-plugin-core/src/layer	Request coordination	Mixed	Actor / message-passing model	Arc,Mutex,Condvar	ACK-based backpressure improves flow control and prevents overload, but lifecycle coordination between writers, readers, and manager state is complex
nushell/nushell:crates/nu-protocol/src/engine/layer	Runtime coordination	Mixed	Shared-state concurrency model	Arc,Mutex,AtomicBool,AtomicU32, Arc::make_mutex, Send/Sync	Layered coordination combines shared immutable metadata with mutex-protected runtime services, but the large central runtime state increases coupling and synchronisation complexity
nushell/nushell:crates/nu-protocol/src/pipeline/layer	Event dispatch system	Coordination-heavy	Shared-state concurrency model		Simple shared registry design guarantees correctness during handler execution, but the mutex serialises registration and dispatch operations
nushell/nushell:crates/nu-utils/src/sync/ke	Caching layer	Coordination-heavy	Initialisation / singleton coordination model	RwLock, OnceLock, LazyLock	Read-heavy cache scales efficiently through RwLock and per key OnceLock initialisation, but insertion and initialisation paths remain coordination-heavy
sharkdp/fd:src/main.rs	Runtime coordination	CPU-bound	Initialisation / singleton coordination model	Arc	Immutable shared configuration minimises synchronisation overhead, but this layer only bootstraps downstream threadpool coordination
sharkdp/fd:src/walk.rs	I/O coordination layer	I/O-bound	Producer-consumer / pipeline coordination model	Arc,Mutex,AtomicBool	Batching and bounded queues improve filesystem traversal throughput, but cancellation flags and batch ownership coordination add implementation complexity
sxyazi/yazi:yazi-core/src/tasks/tasks.rs	Runtime coordination	Coordination-heavy	Shared-state concurrency model	Arc	Uses snapshotting pattern, minimises lock duration, avoids holding locks during UI updates, relies on Arc for shared ownership instead of channels
sxyazi/yazi:yazi-dds/src/pubsub.rs	Event dispatch system	Coordination-heavy	Shared-state concurrency model	RwLock	Registry as routing table, snapshot, no long held locks, event dispatch pipelines
sxyazi/yazi:yazi-dds/src/server.rs	Connection	I/O-bound	Actor / message-passing model	RwLock	Shared-memory: directory, message-passing:communication, use of snapshot, lock minimised /never held across async IO, broadcast via cloned channel senders
sxyazi/yazi:yazi-runner/src/loader/loader.rs	Caching layer	I/O-bound	Initialisation / singleton coordination model	RwLock	caching (Rust cache + Lua package.loaded), lazy binary compilation, optimised for many readers and rare writers
sxyazi/yazi:yazi-scheduler/src/scheduler.rs	Runtime coordination	Coordination-heavy	Producer-consumer / pipeline coordination model	Arc	Arc is for system level coordination, most data flow is message-passing, Arc is used for shared engines and registries, ID avoids direct shared mutable structs
sxyazi/yazi:yazi-scheduler/src/worker.rs	Resource coordination	Mixed	Worker-thread pool execution model	Arc,parking_lot::Mutex	Separation between execution and coordination: work is message-passed, lifecycle/cancellation/progress are shared memory, avoids holding locks across .await, subsystem sharing

firecracker-microvm/firecracker/src/firecracker	Request coordination	Coordination-	Actor / message-	arc,mutex	API requests are isolated from the VM and sent through message passing to the VMM thread, which keeps VM state safe, but adds overhead and delays because all changes must go through a queue and event system.
	layer	heavy	passing model		
firecracker-microvm/firecracker/src/firecracker	Runtime coordination	Coordination-	Actor / message-	arc,mutex	The main process coordinates VM, metrics, and threads using shared state and event loops, which keeps structure clear, but increases coupling and requires synchronisation across many components
	layer	heavy	passing model		
firecracker-microvm/firecracker/src/vmm/s	Runtime coordination	Coordination-	Initialisation /	arc,mutex	VM setup shares state across components using Arc and Mutex so everything can access the same VM, but this increases coupling between initialisation and runtime systems
	layer	heavy	singleton coordination model		
firecracker-microvm/firecracker/src/vmm/s	Runtime coordination	Mixed	Actor / message-	arc,mutex,barrier	Event-driven design with threads for vCPUs and devices, which improves performance by separating execution and control, but requires complex coordination using channels, barriers, and event signals
	layer		passing model		
firecracker-microvm/firecracker/src/vmm/s	OS integration bridge	Mixed	Worker-thread pool	arc,barrier	Each vCPU runs in its own thread and communicates through message passing, which enables parallel execution, but adds complexity for synchronisation and coordination between threads
			execution model		
firecracker-microvm/firecracker/src/vmm/s	Resource coordination	Mixed	Shared-state	arc,mutex,AtomicU32	Shared VM state is accessed by many threads using locks and atomics, which keeps memory consistent, but introduces performance overhead and increases complexity
	layer		concurrency model		
casey/just/src/analyzer.rs	Shared configuration	CPU-bound	Initialisation /	arc	Static analysis over immutable shared data makes execution simple and fast, but it assumes everything never changes during analysis
	system		singleton coordination model		
casey/just/src/dependency.rs	Shared configuration	CPU-bound	Initialisation /	arc	Very lightweight representation of dependencies keeps the system simple and fast, but only supports static traversal and formatting
	system		singleton coordination model		
casey/just/src/justfile.rs	Runtime coordination	Mixed	Resource arbitration	Arc, mutex	Parallel execution of dependencies improves throughput, but requires locking to prevent duplicate execution, which adds overhead
	layer		model		
casey/just/src/ran.rs	State synchronisation	Coordination-	Resource arbitration	arc,mutex	Central registry prevents the same work from running twice, which ensures correctness in parallel execution, but the global lock can become a bottleneck when many tasks are started at once
	layer	heavy	model		
casey/just/src/signal_handler.rs	OS integration bridge	Coordination-	Shared-state	mutex	Signal handler keeps system-wide behavior consistent, but all signal updates go through a single lock, which limits parallel handling
		heavy	concurrency model		